Latent Reasoning Interpretability in CODI on GSM8K

Grad NLP Final Project

Karen Li, Sandra Luo, Hannah Tao

PROBLEM STATEMENT

- **Motivation.**
  CoT prompting improves multi-step reasoning but at significant inference cost. CODI distills CoT into a fixed K=6 latent loop — no explicit token-level reasoning chain.
- **Setup.**
  CODI-GPT-2 (small) trained on GSM8K. K=6 latent thoughts between <BOT> and <EOT>. The shared LM is both teacher (text CoT) and student (continuous latents). Each latent is the prior step's last-token hidden state fed back as an embedding (no decoding).
- **Problem.**
  Latent reasoning is unreadable. If models reason this way at scale, we lose the audit-trail that explicit CoT provides.
- **Research questions.**
  RQ1: What is the role of the six latent thoughts?
  RQ2: How does the latent model compute the answer?
  RQ3: Can we causally control its output via single-cell steering?

METHODS

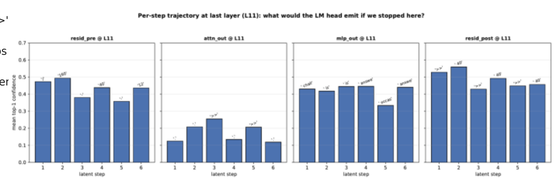
- **Setup.**
  Model: public CODI-GPT-2 checkpoint, 12 blocks × 12 heads, H=768, K=6 latent steps; greedy decoding; no fine-tuning.
- **Benchmarks.**
  GSM8K test (N=1319). Auxiliary CF datasets we built to disentangle operators from operands: gsm8k_vary_operator (template-controlled), gsm8k_cf_op_strict, gsm8k_cf_natural.
- **Forced decoding.**
  Cap the latent loop at step k ∈ {1..6}, decode, and measure accuracy. Captures how much each latent step actually contributes to the answer.
- **Multi-op probes.**
  RidgeClassifier per (step, layer, marker position) predicting marker m's operator or last-digit. Class-balanced; shuffle-step null baseline.
- **Logit lens.**
  Apply final LayerNorm + LM head at every (step, layer, sublayer) hidden state. Track which token is being predicted at every cell of the loop.
- **Activation patching + linear steering.**
  Paired-CF interchange / zero / mean ablation at (step, layer, block). Linear steering: add α·v to residual or attention output at probe-winning cells, with v ∈ {LDA, vary-op template-difference, Adam-trained, LM-head row-difference}. Sweep α and measure operator-emit flip rate.
- **Correctness probe.**
  Per-(step, layer) RidgeClassifier predicting whether the final force-decoded answer will be correct. Tests whether the model's residuals encode its own success / failure.

RESULTS — CODI-GPT-2 on GSM8K

RQ1: Role of the six latent thoughts

Even/odd storage vs computation.

Logit-lens at L11 reveals a striking alternating rhythm. Odd steps (1, 3, 5) produce '>>' last layer; even steps → The latent loop liter

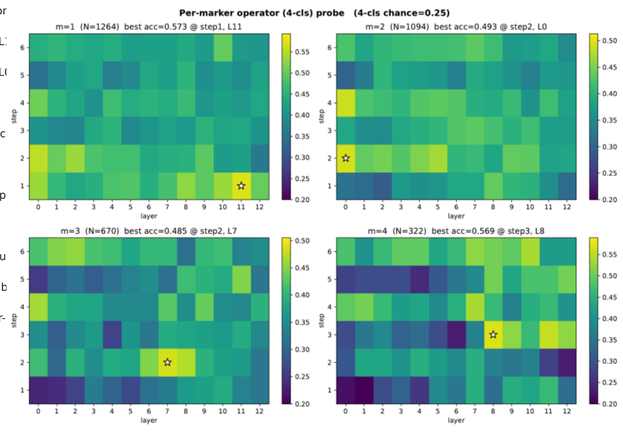


Logit-lens at last layer L11 — modal tokens per latent step. Odd steps commit to '>>'; even steps to digits.

Operator info is highly decodable from CODI's residuals.

4-class RidgeClassifier (`+, −, ×, ÷`) trained per (step, layer) cell to predict marker m's gold operator from the last-token residual. m=1..4 = position in the gold marker chain (<<a op b = c>>). Class-balanced; stratified 80/20 split per cell. Chance = 0.25.

Best-cell acc or
m=1: step1/L
m=2: step2/L
Filter to CODI-c
m=1: 0.812
→ The clean op
Shuffle-step nu
3 seeds): real b
carries marker-



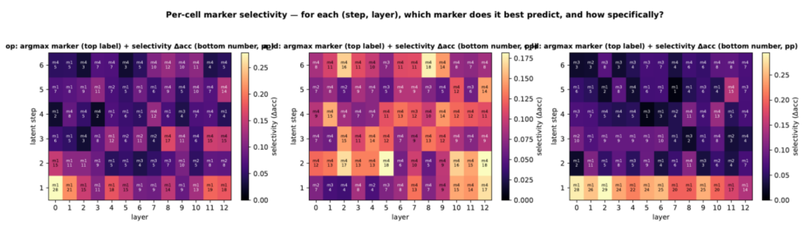
Operator probe heatmaps per marker (4 panels). Bright cells = high probe accuracy. Best cells starred per marker.

Step ordering carries marker-position info.

Shuffle-step null baseline: per-example random permutation of the 6 step indices. Real beats shuffled by Δ +0.07-0.17pp on op and c_ld probes. Step index, not just step content, matters per marker.

Cross-marker selectivity.

Best (step, layer) cells are weakly marker-specific (diagonal advantage +0.04 to +0.15pp). Cells encode operator structure generically with mild per-marker specialization.



Per-cell argmax-marker + selectivity (Δ acc). c_ld concentrates on odd steps (1, 3, 5) for all markers.

Attention writes the LM-head-aligned tokens; MLP role is opaque.

Logit-lens per sublayer: attn-out at L9-L11 commits to ' *' (step 1 L10, 0.58 conf), ' <<' (step 3 L10, 0.53), digits (even steps). MLP-out at the same cells projects to non-vocab-aligned directions — typical for transformer MLPs; doesn't mean they're 'noise', just that their contribution isn't readable via logit lens. Single-cell mean-ablations of either block change acc by <1.3pp (max), so the compute is distributed; neither block alone is causally critical at any cell.

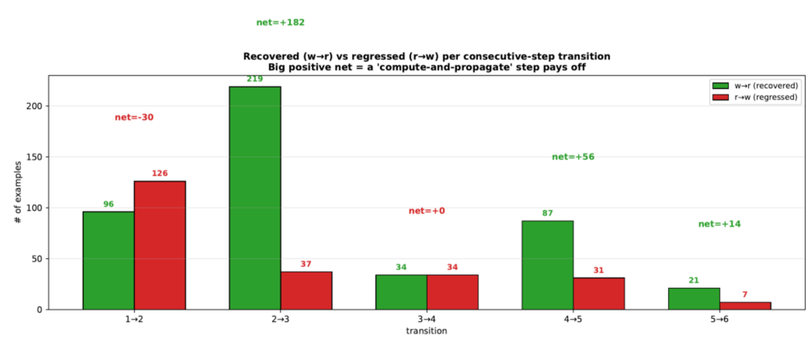
RQ2: How does the model compute?

Two calculation windows.

Force-decode acc per step: 25%→23%→37%→37%→41%→42%. Biggest jump is step 2→3 (+13.8pp); a second jump at 4→5 (+4pp). Step 1→2 is actually a net REGRESSION (−30 net w↔r transitions).

Commit-then-synthesize rhythm.

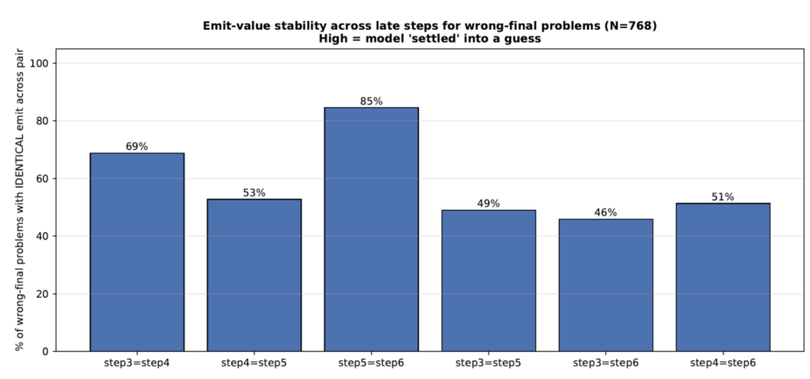
Step 2 has the loop's largest MLP write (norm 564 vs 419-535 elsewhere). Step 3 then attends back to L2 (13% — the loop's strongest step→prior-latent attention). Step 4→5 repeats it (L4 →



Per-example w→r / r→w transitions per consecutive step. Step 1→2 is a NET REGRESSION (−30); step 2→3 rescues +182.

After step 3, almost no recovery.

Of 838 problems wrong at step 3, only 117 (14%) recover by step 6. Wrong-final emits then stabilize: 48% have ONE unique value across steps 4-6; 93% have ≤ 2. Approximation mode kicks in.



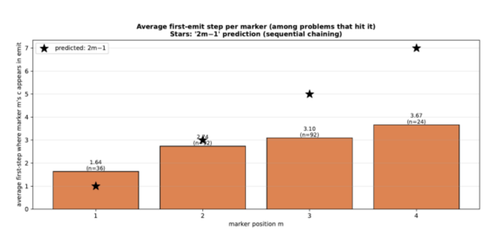
Recovery rate by 'first-wrong' step (left); late rescue dominated by longer chains (right, n_markers stacked).

Step chaining — sequential but compressed.

Among Mode 1 (loop-rescued) ≥2-marker problems: avg first-emit step grows monotonically with marker position — m=1→1.64, m=2→2.74, m=3→3.10, m=4→3.67. Chain order is real (9/10 monotone for 2-marker chains); gaps are compressed.

Mostly INTERNAL chaining.

67% of Mode 1 problems emit only ONE marker's c-value (usually the final). The chain happens in residuals, not in emits.



Mode 1 chaining: avg first-step per marker grows monotonically. Black stars = '2m−1' sequential prediction; data is compressed.

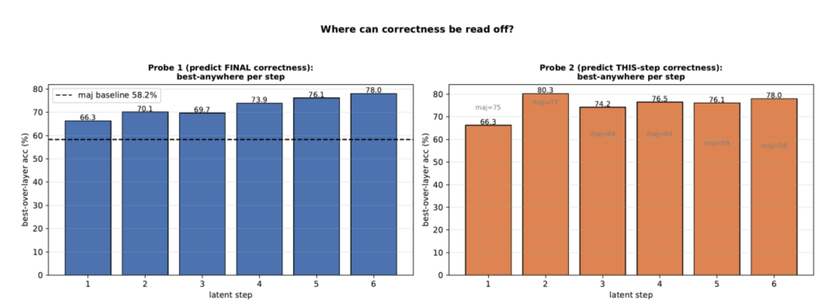
Correctness — compute or approximate?

Two operating regimes.

CODI either (1) computes via the loop, or (2) approximates with a Benford-shaped guess. 551/1319 correct (41.8%) split as 280 by loop work (Mode 1) and 271 already at step 1 (Mode 2 — shortcut).

Correctness probe — the model knows when it's wrong.

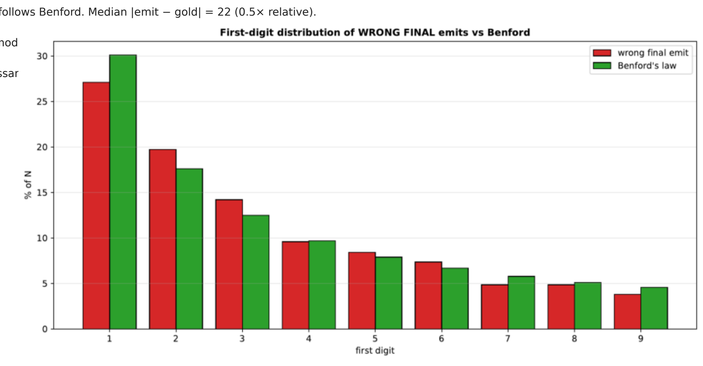
Per-(step, layer) RidgeClassifier on the final-emit correctness label rises 66% → 78% (step 1 → step 6). Majority baseline = 58% → +20pp at step 6; +8pp already at step 1 (pre-compute signal).



Correctness probe: best-over-layer acc per step. Probe 1 (predict final) rises 66% → 78%; probe 2 (this-step) similar.

Wrong emits match gold's distribution.

Final wrong answers: 37% end in 0, 45% mult-of-5 (gold: 46%); first-digit follows Benford. Median |emit − gold| = 22 (0.5× relative). The mod necessar



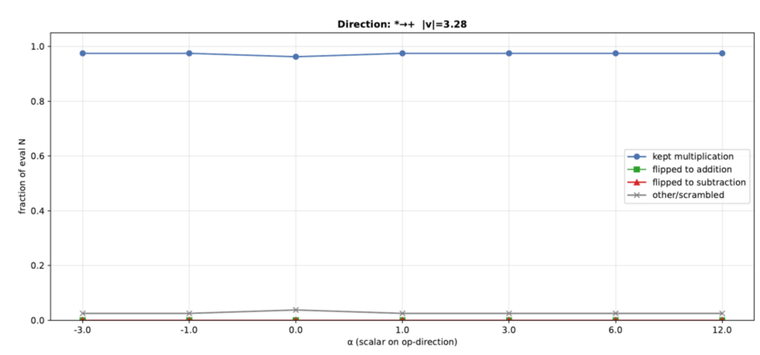
Top wrong-final values. Most-common: 2, 4, 30, 5, 20, 60. Wrong answers are Benford-shaped natural-looking numbers.

Operation steering has NO effect (7 experiments).

At probe-winning cells, add α·v to residual / attn-out. Vectors: LDA centroid, vary-op template-diff, Adam-optimized, LM-head row-diff. ~5000 steered runs at α magnitudes up to 100× direction norm. ZERO operator flips. Trained v makes loss drop 21.5→13.6 but argmax doesn't move. Probes find structure; steering doesn't.

Why? — 72-91% of attention goes back to Q.

The latent loop is mostly question re-encoding. Modifications at any single cell get washed out by the next step's re-attention to the question tokens.



Last-attempt steering using the LM head's OWN row-difference at the cell where attn-out modal token is '': 0 flips across α ∈ [−3, +12].*

Take-home: probes find rich operator/marker/correctness signal in CODI's residuals (op 0.80–0.90, correctness 78%, +20pp baseline) — but linear single-cell steering can't move the emit. Compute rhythm = step 2 commit → step 3 synthesize → step 4 commit → step 5 synthesize → approximation (Benford-shaped wrong guesses).